Foundations of Computer Programming: Introduction to object-oriented programming

Learning objectives:

- Understand programming and its fundamental concepts.
- Understand different programming languages and recognise the difference
- Understand characteristics of High-level and Low-level Programming Languages
- Understand the Rules of a Language (Syntax & Semantics)
- Develop a Comprehensive Understanding of Object-Oriented Programming
- Recognise the distinction between Procedural and Object-Oriented Programming approaches.
- Understand the fundamental principles of object-oriented programming

Lesson exercise: Week 1

Answer the following questions individually. Once you have completed this task, you will collaborate with a group of 3-5 students to create a presentation on "Object-oriented programming, four OOP principles, and the history of Python".

1. What is Computer programming?

   Computer programming is the process in which professionals write code to direct the operations and functionality of a computer, application, or software program. Computer programming is a set of instructions that direct a computer's actions.

2. What are the fundamental concepts of computer programming?

   The fundamental concepts of computer programming include variables, data types, operators, control structures, functions, and input/output operations.

3. What are the different types of programming languages?

   The main categories of programming languages are low-level (machine or assembly language) and high-level (such as Python, Java, C++).

   Low-level languages, such as assembly language, are close to the computer's machine language and require a deep understanding of the computer's hardware and architecture.

   High-level languages, such as Python, Java, and C++, are closer to human language and are more abstracted from the computer's hardware.

4. What are the characteristics of high-level and low-level programming languages?

   High-level programming languages: Closer to human language, more abstracted from hardware, easier to read and write, portable across different computer architectures.

   Low-level programming languages: Closer to machine language, less abstracted from hardware, harder to read and write, not portable across different computer architectures.

5. What is machine language?

Machine language is the lowest-level programming language, represented as binary code that the computer's processor can directly execute.

6. What is assembly language?

Assembly language is a low-level programming language that provides a human-readable representation of the underlying machine language.

7. What is object-oriented programming?

Object-oriented programming is a programming paradigm that uses objects, which are instances of classes, to encapsulate data and the methods that operate on that data.

The four fundamental principles of object-oriented programming are,

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

8. What are the different concepts in object-oriented programming and explain them?

The four fundamental principles of object-oriented programming are:

- Encapsulation: Hiding the internal implementation details of an object and providing an interface to interact with the object.
- Abstraction: Focusing on the essential features and behaviours of an object, hiding unnecessary details.
- Inheritance: Creating new classes based on existing ones, inheriting their properties and methods.
- Polymorphism: The ability of objects of different classes to respond to the same method call in different ways.

9. What is the difference between procedural and object-oriented programming?

Procedural programming is a programming paradigm that follows a top-down approach, with a series of step-by-step instructions for the computer to execute.

Object-oriented programming is a programming paradigm that focuses on creating objects, which are instances of classes, and organising the code around these objects and their interactions. Object-oriented programming is based on the four principles of encapsulation, abstraction, inheritance, and polymorphism.

The key differences between procedural and object-oriented programming are:

- Procedural programming is focused on functions and procedures, while object-oriented programming is focused on objects and classes.
- Procedural programming follows a top-down approach, while object-oriented programming follows a bottom-up approach.
- Procedural programming is based on a sequence of instructions, while object-oriented programming is based on the interactions between objects.

10. Why it is important to understand syntax and semantics?

Understanding syntax and semantics is crucial for writing effective and correct computer programs. Syntax refers to the rules and grammar of a programming language, such as how variables are declared, how statements are structured, and how functions are defined. Following the syntax rules is necessary for the computer to correctly interpret and execute the code. Semantics refers to the meaning and interpretation of the code, ensuring that the instructions and logic are correctly implemented to achieve the desired outcome.

Proper understanding of syntax and semantics helps developers to write clean, maintainable, and efficient code, avoiding syntax errors and logical mistakes.

11. What is the difference between compiled and interpreted programming languages?

Compiled languages like C and C++ are first translated into machine code and then executed, while interpreted languages like Python and JavaScript are executed line by line without being fully compiled first.

The key differences:

- Compilation process: Compiled languages go through a separate compilation step, while

interpreted languages are executed directly.

- Execution speed: Compiled programs generally run faster, as the machine code is optimized for the specific hardware.

- Portability: Interpreted languages are more portable, as the same code can run on different platforms without recompilation.

The choice between compiled and interpreted languages depends on the specific requirements of the project, such as performance, development speed, and portability.

12. The history of Python:

Python was created in the late 1980s by Guido van Rossum at the Netherlands National Research Institute for Mathematics and Computer Science.

Python is a high-level, interpreted, general-purpose programming language that emphasises code readability and simplicity, making it a popular choice for scientific computing, robotics, and web development.

Python's popularity has grown significantly in recent years, particularly in the field of machine learning, where it has become a dominant language due to its extensive library support and ease of use.